

divide the module into multiple files. Let us understand the procedure of building a module that spans two files. Let's divide the initialization and cleanup functions from the *hello.c* file into two separate files, namely *startup.c* and *cleanup.c*.

Given below is the source code for *startup.c*:

```
#include <linux/kernel.h>
#include <linux/module.h>

static int __init hello_init(void)
{
    printk(KERN_INFO "Function: %s from %s file\n", __func__,
    __FILE__);

    return 0;
}

module_init(hello_init);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Narendra Kangralkar.");
MODULE_DESCRIPTION("Startup module.");
MODULE_VERSION("1.0");
```

And "cleanup.c" will look like this.

```
#include <linux/kernel.h>
#include <linux/module.h>

static void __exit hello_exit(void)
{
    printk(KERN_INFO "Function %s from %s file\n", __func__,
    __FILE__);
}

module_exit(hello_exit);

MODULE_LICENSE("BSD");
MODULE_AUTHOR("Narendra Kangralkar.");
MODULE_DESCRIPTION("Cleanup module.");
MODULE_VERSION("1.1");
```

Now, here is the interesting part -- *Makefile* for these modules:

```
obj-m += final.o
final-objs := startup.o cleanup.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD)
```

clean

The *Makefile* is self-explanatory. Here, we are saying: "Build the final kernel object by using *startup.o* and *cleanup.o*." Let us compile and test the module:

```
[mickey]$ ls
cleanup.c  Makefile  startup.c

[mickey]$ make
```

Then, let's display module information using the *modinfo* command:

```
[mickey]$ modinfo final.ko
filename:        final.ko
version:         1.0
description:     Startup module.
author:          Narendra Kangralkar.
license:         GPL
version:         1.1
description:     Cleanup module.
author:          Narendra Kangralkar.
license:         BSD
srcversion:      D808DB9E16AC4004780E2F
depends:
vermagic:        2.6.32-358.el6.x86_64 SMP mod_unload
modversions
```

Here, the *modinfo* command shows the version, description, licence and author-related information from each module.

Let us load and unload the *final.ko* module and verify the output:

```
[mickey]$ su -
Password:

[root]# insmod final.ko

[root]# dmesg
Function: hello_init from /home/mickey/startup.c file

[root]# rmmod final

[root]# dmesg
Function: hello_init from /home/mickey/startup.c file
Function hello_exit from /home/mickey/cleanup.c file
```

Passing command-line arguments to the module

In user-space programs, we can easily manage command line arguments with *argc/argv*. But to achieve the same functionality through modules, we have to put in more of an effort.